



MTV Django Architecture

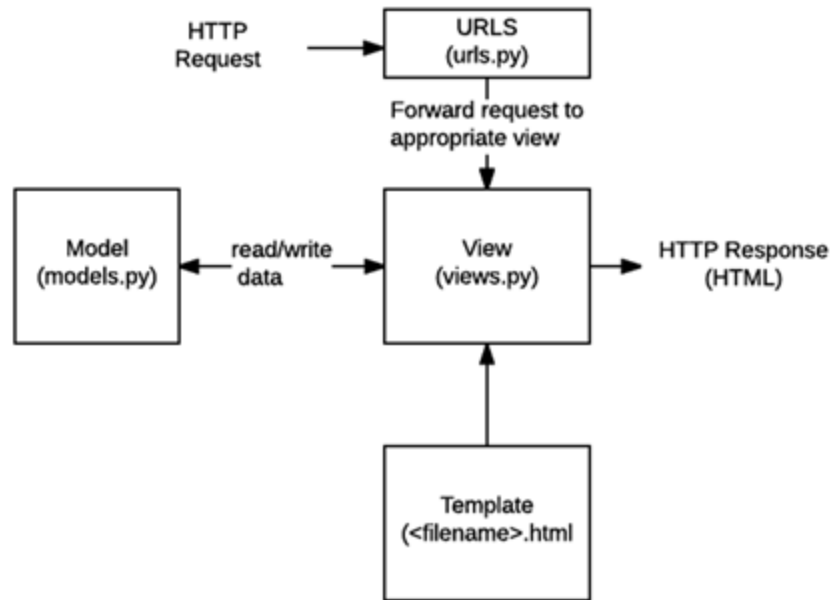
Tim Dosen PBP

MTV Django Architecture

Django MTV Architecture

Django is often referred to as an MTV framework

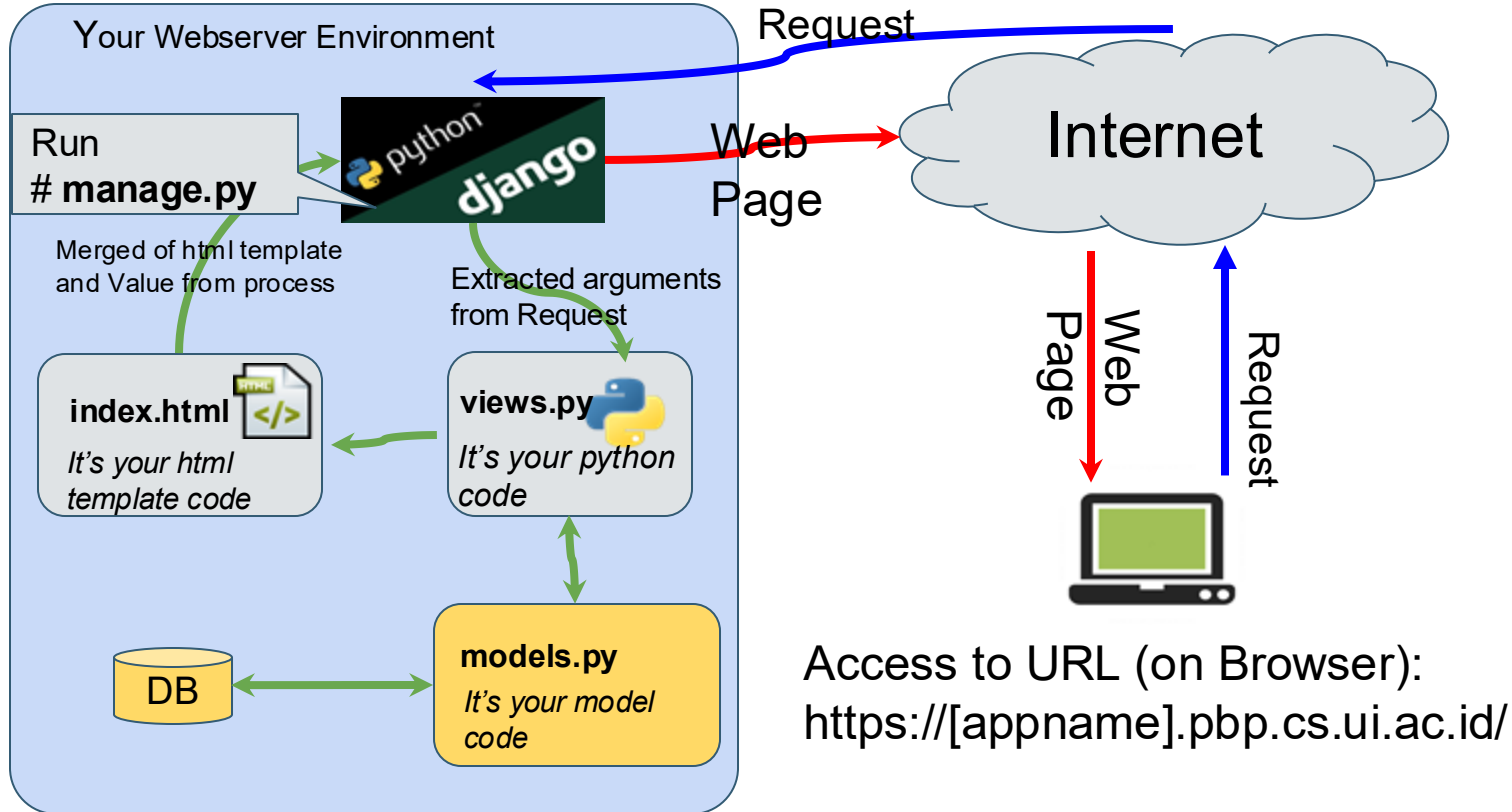
- **M** stands for “**Model**”: the data access layer
- **T** stands for “**Template**”: the presentation layer
- **V** stands for “**View**”: the business logic layer, the bridge between models and templates.



Question

Separation of Concern: Why we need it? What are the benefits of it?

How Django Framework Works



Installing Django

Installing Django

1. Install Python

“Being a Python web framework, Django requires Python”

2. Install and activate a Python Virtual Environment

```
python -m venv env  
env\Scripts\activate.bat
```

3. Install Django and/or required dependencies

```
pip install django
```

Or

```
pip install -r requirements.txt
```

MY LAPTOP



C:\Project\
Lomba\
Eg: Django 1.7

C:\Project\PBP\
MyWeb\
Eg: Django 3.2

PC LAB

D:\Data\PBP\
MyWeb
Eg: Django 4.1.1

Server di Luar Negeri

/home/pbp/
Eg: Django 3.2

Django Project vs Django App

Question

What is the difference between Django project and Django app?

Django Project vs Django App

- **Django Project**

- A Django project – a collection of settings for an instance of Django, including database configuration, Django-specific options and application-specific settings.
- A project is a collection of configuration and apps for a particular website.
- A project can contain multiple apps.

- **Django App**

- An app is a web application that does something – e.g., a blog system, a database of public records or a small poll app.
- An app can be in multiple projects.



project name: pbp2021

dj project



- `_init_.py`
- `settings.py`
- `urls.py`
- `wsgi.py`
- `asgi.py`

dj apps



- apps_1
 - apps_2
 - apps_3
 - ...



`/templates`
`apps_1/templates/`
`index.html.etc`



- `_init_.py`
- `admin.py`
- `apps.py`
- `models.py`
- `views.py`
- `urls.py`
- `tests.py`

request +
response



the DB

Django Project

- Start a project

```
django-admin startproject contohproject
```

- Auto-generated files:

contohproject/

contohproject/

__init__.py

asgi.py

settings.py (configuration for the project)

urls.py (starting point to configure urls)

wsgi.py

manage.py (command runner)

- For other django-admin commands, see Django documentation.

Django Development Server

- Start up a development server

```
cd contohproject  
python manage.py runserver
```

- The Django development server is meant to be used only during development so you can develop things rapidly, without having to deal with configuring a production server – such as Apache – until you're ready for production.
- By default, the runserver command starts the development server on the internal IP at port 8000.
- After the server's running, visit <http://127.0.0.1:8000/> or <http://localhost:8000/> with your web browser.

Django App

- Build an app

```
django-admin startapp contohapp
```

- The directory structure of the newly created **contohapp** application:

```
contohapp/  
    migrations/  
        __init__.py  
    __init__.py  
    admin.py  
    apps.py  
    models.py  
    tests.py  
    views.py
```

Django App

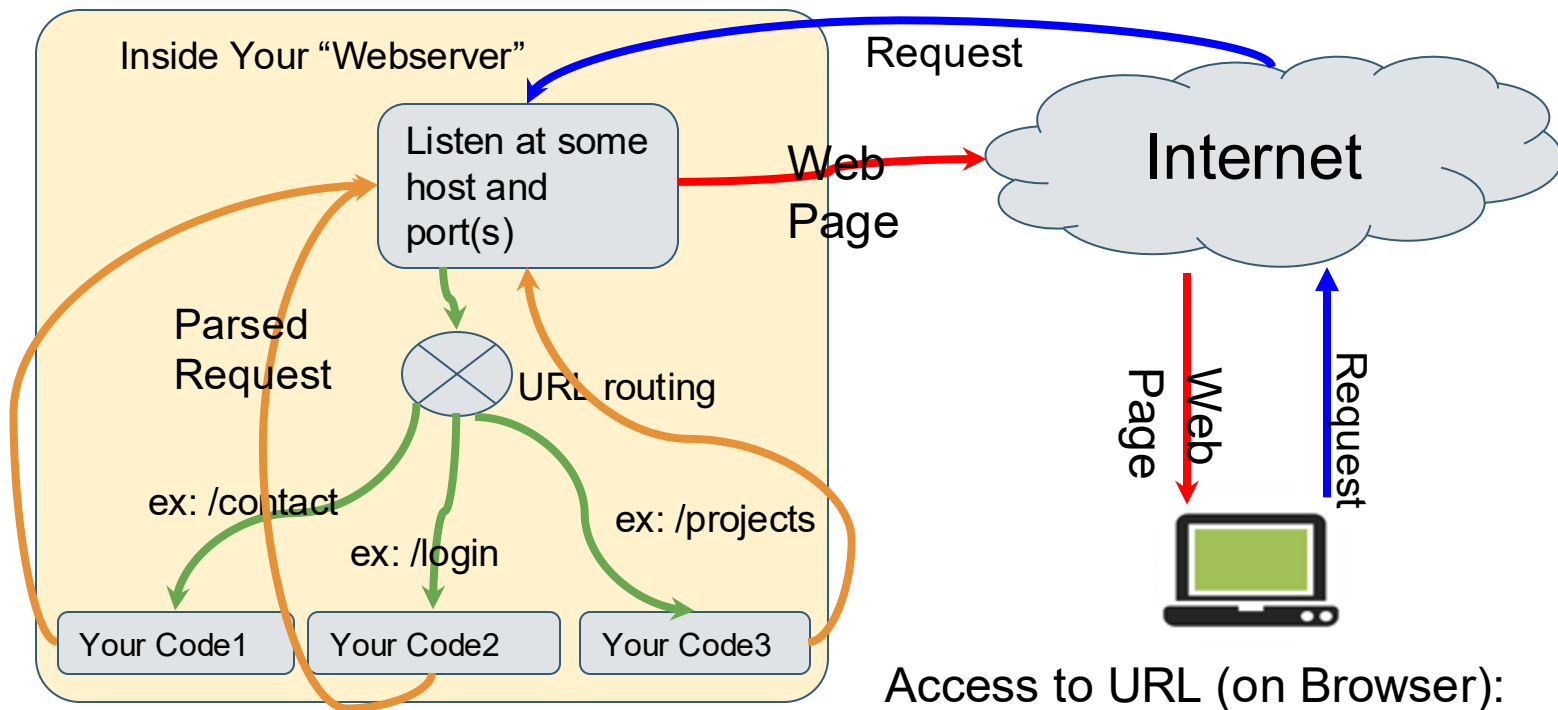
- We have to tell Django that it should use the `contohapp` application.
- Open `contohproject/settings.py` and add `contohapp` in the `INSTALLED_APPS` setting,

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'contohapp',  
]
```

Baca di Django documentation, apa saja aplikasi yang secara default sudah ada di `INSTALLED_APPS` setting dan apa fungsi dari masing-masing aplikasi tersebut.

Django URLs and Views

Django URLs and Views



Access to URL (on Browser):
`https://[appname].pbp.cs.ui.ac.id/`

Django URLs

- Django URLs forward request to appropriate view
- Django maps arguments from request or url to a view by using regular expression
- Contoh: supaya dapat mengakses **contohapp** di <http://localhost:8000/contohapp>
 1. Buatlah berkas `urls.py` di direktori aplikasi **contohapp**
 2. Tulis di dalam berkas tersebut:

```
from django.urls import include, path
from .views import index

urlpatterns = [
    path('', index, name='index'),
]
```

Django URLs

3. Di dalam berkas `urls.py` yang ada direktori project `contohproject`, import `django.urls.include` dan tambahkan modul `contohapp.urls` di dalam variabel `urlpatterns` sehingga isi berkas `urls.py` menjadi:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('contohapp', include('contohapp.urls')),
]
```

Django Views

- A view function, or view for short, is a Python function that takes a web request and returns a web response. This response can be the HTML contents of a web page, a redirect, a 404 error, an XML document, an image, or anything.
- Untuk **contohapp**, setelah membuat konfigurasi urls, selanjutnya adalah buka berkas `views.py` yang ada di direktori **contohapp** dan tambahkan kode berikut ini:

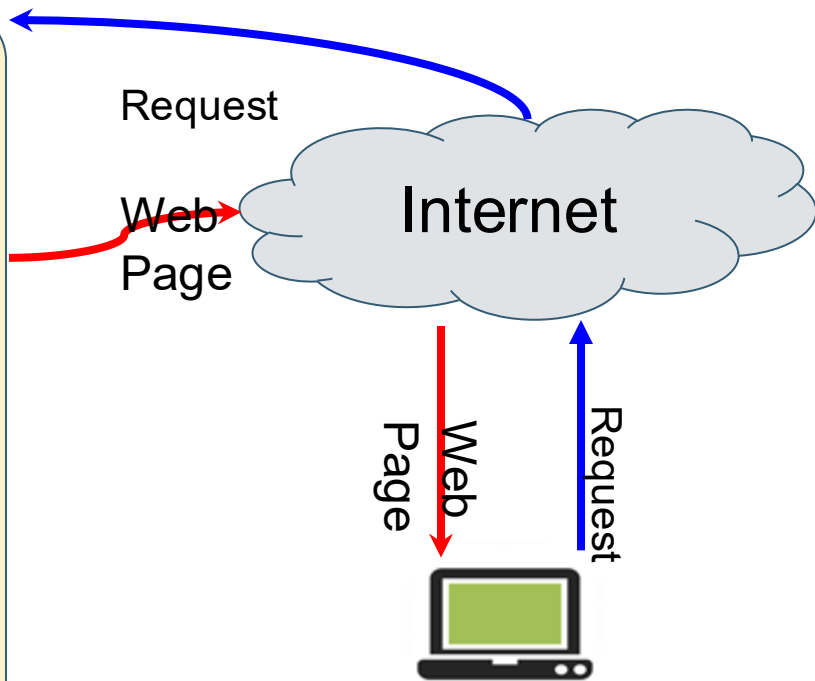
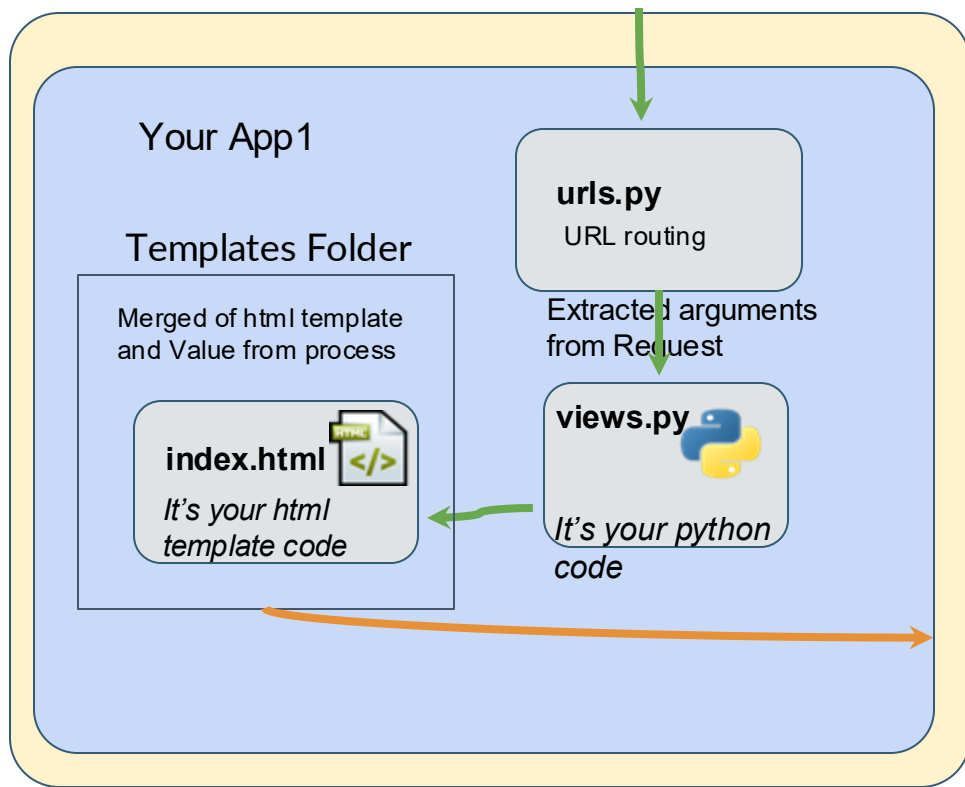
```
from django.http import HttpResponseRedirect

def index(request):
    return HttpResponseRedirect ("Hello world!")
```

- Sekarang seharusnya <http://localhost:8000/contohapp> sudah dapat diakses di web browser.

Django Views and Templates

Django Views and Templates



Access to URL (on Browser):
[https://\[appname\].pbp.cs.ui.ac.id/](https://[appname].pbp.cs.ui.ac.id/)

Django Templates

- Most of frameworks include template engines
- Our apps likely has a lot of html that doesn't change (ex: header and footer) and some that does (ex: body)
- By convention DjangoTemplates looks for a "templates" subdirectory in each of the INSTALLED_APPS.
- Subdirektori "templates" harus kita buat sendiri.

Untuk **contohapp**, buatlah folder bernama "templates" di dalam direktori aplikasi **contohapp** dan di dalam folder tersebut, buatlah berkas `index.html` berisi kode berikut ini:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{ name }}</title>
</head>
<body>
  <h1>Hello My Name is {{ name }}</h1>
</body>
</html>
```

Django Views and Templates

Views

Update **contohapp**/views.py menjadi:

```
from django.shortcuts import render

mhs_name = 'Mahasiswa PBP'

def index(request):
    response = {'name' : mhs_name}
    return render(request, 'index.html', response)
```

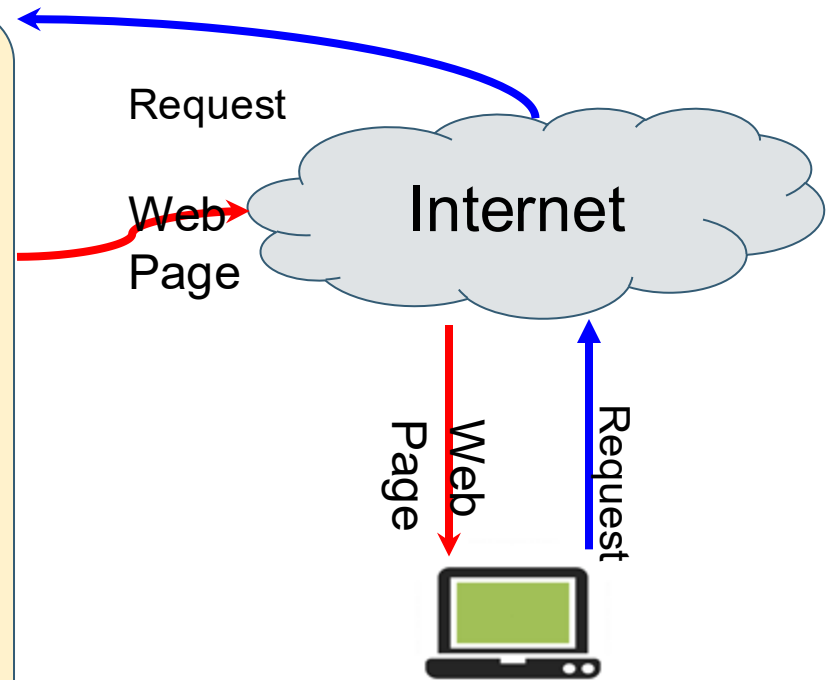
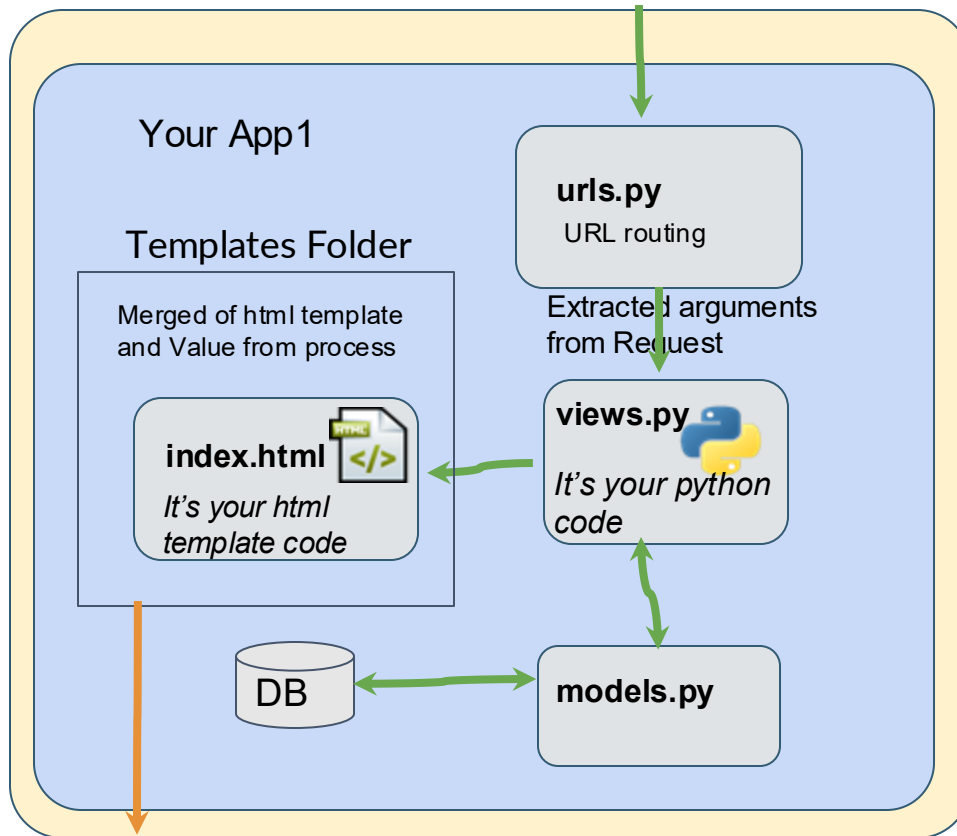
Templates

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{{ name }}</title>
</head>
<body>
    <h1>Hello My Name is {{ name }}</h1>
</body>
</html>
```

Coba cek <http://localhost:8000/contohapp> di browser

Django Views and Models

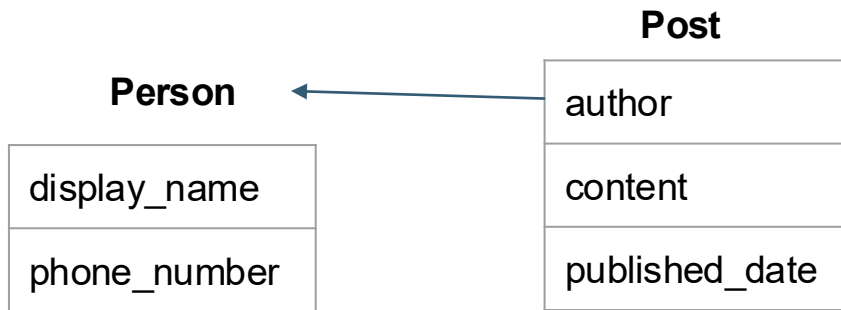
Django Views and Models



Access to URL (on Browser):
[https://\[appname\].pbp.cs.ui.ac.id/](https://[appname].pbp.cs.ui.ac.id/)

Django Models

- Models are a set of data stored to be processed in our apps. We store them in form of tables that connect to one another. We can create, read, update, and delete (CRUD) data from the tables using several specific command instructions called SQL.
- Do you know or remember what is object?



- A model class == a database table
- A model instance == a database table row

Person	
display_name	phone_number
Kak PeBePe	+628123456
Budi	+6281676732

Create Django Models

Update `contohapp/models.py`:

```
from django.db import models
from django.utils import timezone
from datetime import datetime, date

class Person(models.Model):
    display_name = models.CharField(max_length=30)
    phone_number = models.CharField(max_length=17)

class Post(models.Model):
    author = models.ForeignKey(Person, on_delete = models.CASCADE)
    content = models.CharField(max_length=125)
    published_date = models.DateTimeField(default=timezone.now)
```

Django Models - Field

Important part of database is to define the type of each attribute of a class

- `models.CharField` - define text with limited number of chars
- `models.DateField` - a date only
- `models.DateTimeField` - a date and time
- `models.ForeignKey` - a link to another model
- etc ..

Django Models

- Make sure your app is already installed! See settings.py again
- Run:

```
python manage.py makemigrations  
python manage.py migrate
```

Kenapa makemigrations dan migrate dipisah menjadi dua commands?
Apa yang terjadi jika kita hanya menjalankan makemigrations tanpa migrate?

- The three-step guide to making model changes:
 1. Change your models (in models.py)
 2. Run `python manage.py makemigrations` to create migrations for those changes
untuk mempersiapkan migrasi skema model ke dalam database Django lokal
 3. Run `python manage.py migrate` to apply those changes to the database
untuk menerapkan skema model yang telah dibuat ke dalam database Django lokal

Django Admin

Django Admin

- Django Admin is an internal application from Django to help developer/web admin for day-to-day operations, browsing data & support
- By default, the admin site url is <http://localhost:8000/admin/>
- To create a user to login with, use the `createsuperuser` command

```
python manage.py createsuperuser
```

(input username, email, and password)

Django Admin - Register

- To register models to Django Admin, update `contohapp/admin.py`:

```
from django.contrib import admin
from .models import Person, Post
```

```
admin.site.register(Person)
admin.site.register(Post)
```

- Open <http://localhost:8000/admin/>
- Add some `Person` objects in Django Admin

Django Models and Views

- Update **contohapp**/views.py:

```
from django.shortcuts import render
from .models import Person, Post

mhs_name = 'Kak PeBePe'

def index(request):
    persons = Person.objects.all().values()

    response = {'name' : mhs_name, 'persons' : persons}
    return render(request, 'index.html', response)
```

Django Models and Templates

- Update **contohapp**/templates/index.html:

```
...
<body>
    {% if persons %}
        {% for person in persons %}
            {{ person.display_name }}
            <br/>
        {% endfor %}
    {% endif %}
</body>
...
```

Coba cek <http://localhost:8000/contohapp> di browser

Django Tests

Question

How do you verify your tutorial/exercise work? Share your steps to the class!

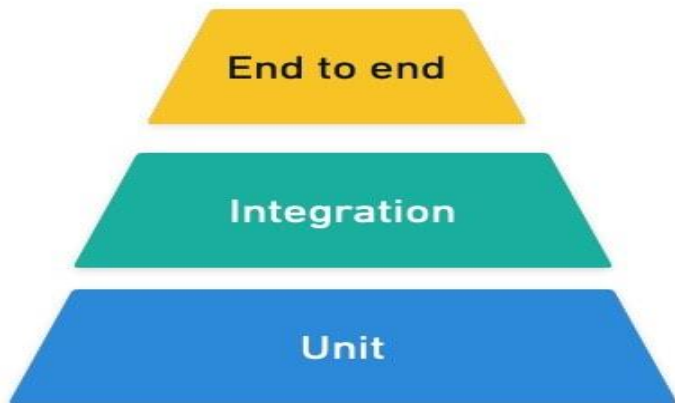
Why Test?



- An example:
 1. Open terminal
 2. Run `python manage.py runserver`
 3. Open web browser
 4. Type `http://127.0.0.1:8000` in the address bar, then send the request
 5. Check if the frontpage displays the expected page
- Manual test **works**, but will get **tedious and repetitive** once the project get more complete
- Even a simple Web app like the one you created in the exercise involve **interactions between multiple components**
 1. E.g., URL routing (`urls.py`) --> request handlers (`views.py`) --> data persistence layer (`models.py`) --> presentation layer/template (the HTML file)
- We want to ensure each component is **correct on their own** and **when interact with other components**

Levels of Testing

The Test Pyramid



Source: <https://semaphore.io/blog/testing-pyramid>

- One approach in developing tests is following the test pyramid scheme
 - Small number of E2E (end-to-end) tests that verify user interactions when operating critical features
 - Medium number of integration tests that verify core component interactions
 - Large number of unit tests to ensure correctness of individual functions/methods
- "Unit" refers to the **smallest unit of building block of software**
 - E.g., individual functions/methods, classes
 - Usually tested in isolation, i.e., without interacting with another unit
- In this course, we will emphasise on **unit tests** and briefly introduce integration & E2E
 - Integration and E2E will be covered more extensively in upper-level programming course, e.g., Advanced Programming in CS and its equivalent course in IS

Django Tests

- Automated testing is an extremely useful bug-killing tool for the modern Web developer. You can use a collection of tests – a test suite – to solve, or avoid, a number of problems:
 - When you're writing new code, you can use tests to validate your code works as expected.
 - When you're refactoring or modifying old code, you can use tests to ensure your changes haven't affected your application's behavior unexpectedly.
- The default `startapp` template creates a `tests.py` file in the new application.

Django Tests Sample

- Update `contohapp`/tests.py:

```
from django.test import TestCase, Client
from django.urls import resolve
from .views import index

class ContohAppTest(TestCase):
    def test_contoh_app_url_is_exist(self):
        response = Client().get('/contohapp')
        self.assertEqual(response.status_code, 200)

    def test_contoh_app_using_person_list_template(self):
        response = Client().get('/contohapp')
        self.assertTemplateUsed(response, 'index.html')
```

Django Tests & Code Coverage

- Run test: `python manage.py test`
- Code coverage
 - Code coverage describes how much source code has been tested. It shows which parts of your code are being exercised by tests and which are not. It's an important part of testing applications, so it's strongly recommended to check the coverage of your tests.
 - To install coverage package: `pip install coverage`
 - To run tests and collect coverage data of the executed files in the project:

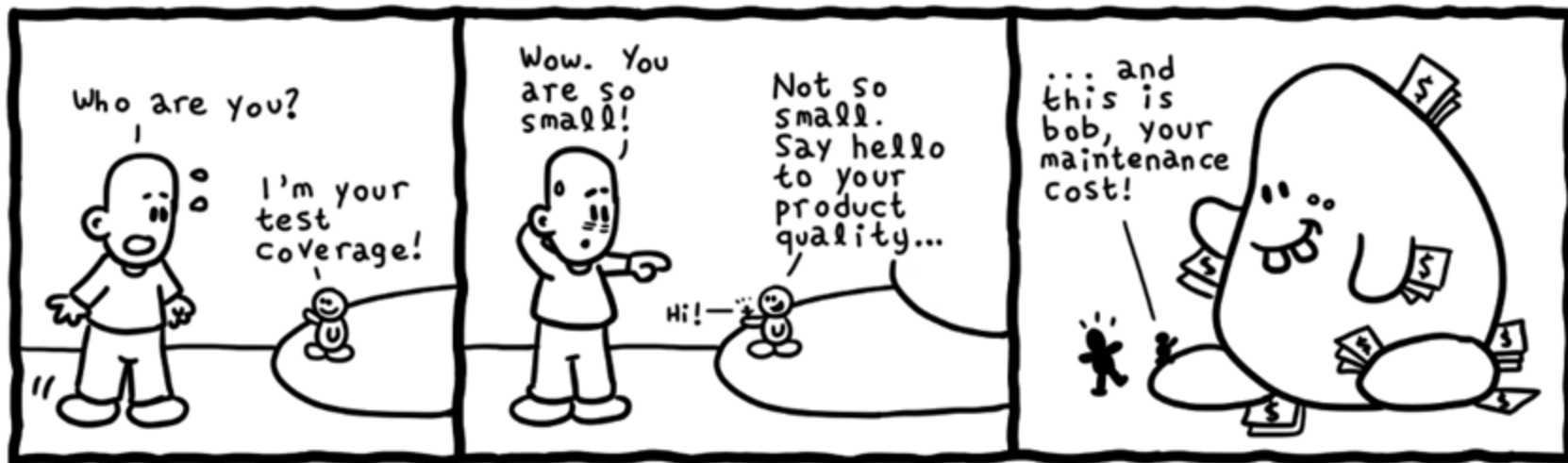
```
coverage run --source='.' manage.py test
```

- To see a report of coverage data:

```
coverage report --show-missing
```

- Note: Code coverage can only indicate that you've forgotten tests; it will not tell you whether your tests are good. Don't use good code coverage as an excuse to write lower quality tests.

Code Coverage



Daniel Stori {turnoff.us}

Code coverage means what percentage of your codebase is covered by the tests or being tested. If you have no tests, then the code coverage is zero.

(<https://codeburst.io/10-reasons-why-code-coverage-matters-9a6272f224ae>)

Additional Materials

Test-Driven Development (TDD)

- TDD is “rediscovered” by Kent Beck.
- TDD is related to the test-first programming concepts of Extreme Programming (one of Agile Software Development methods).

TDD – Unit Tests and Functional Tests

- One of the values and principles of Extreme Programming:
Feedback
 - ❑ Feedback from the system: **unit tests**
 - ❑ Feedback from the customer: **functional/end-to-end tests**
 - ❑ Feedback from the team: When customers come up with new requirements, the team directly gives an estimation of the time that it will take to implement.

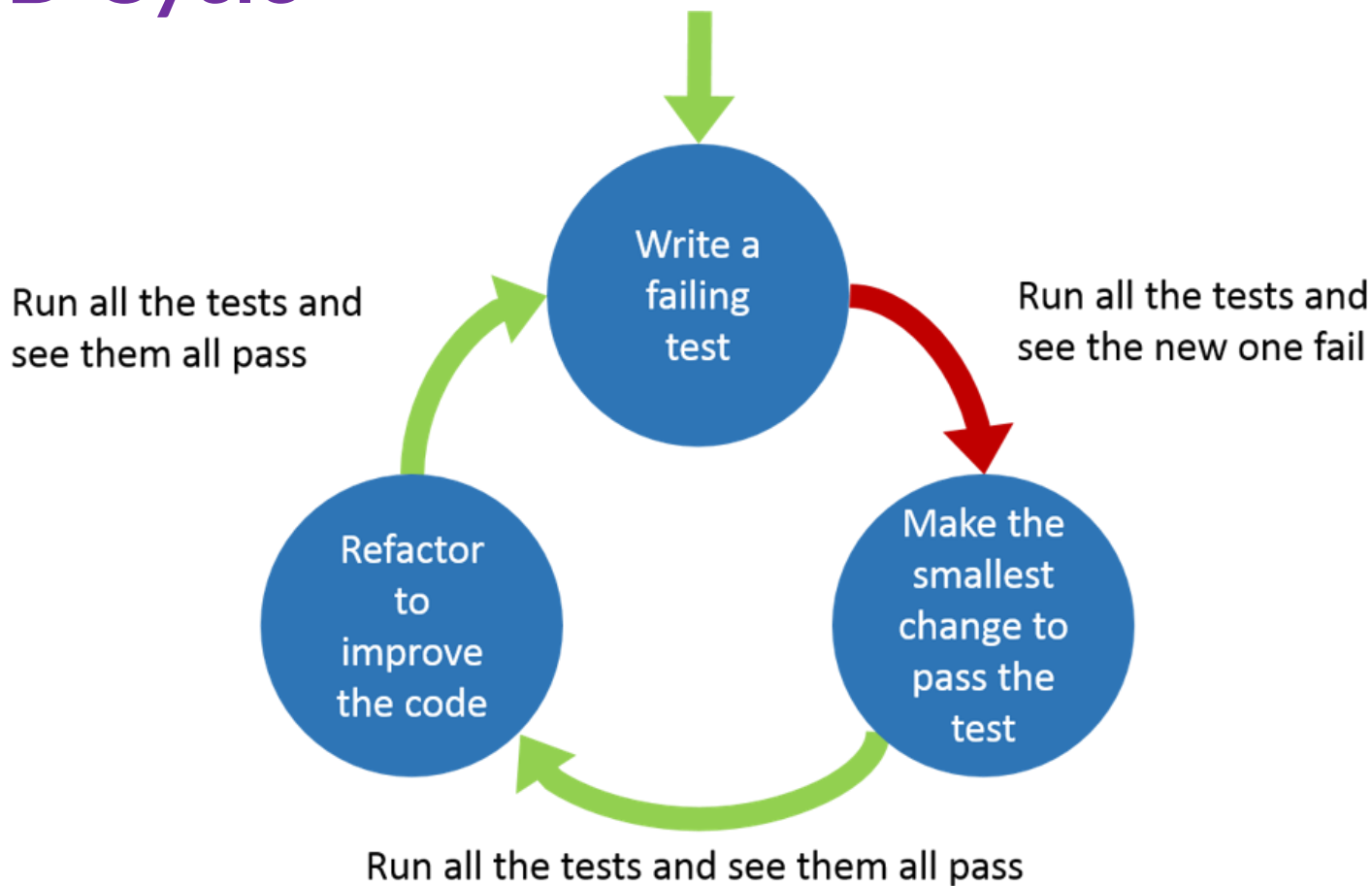
TDD Remarks

- Compare actual result with expected result
- TDD: Think expected result first, write on the test. Later, define the real function that gives the actual result
- Tests are developed FIRST before the code
- Modern TDD: automated testing

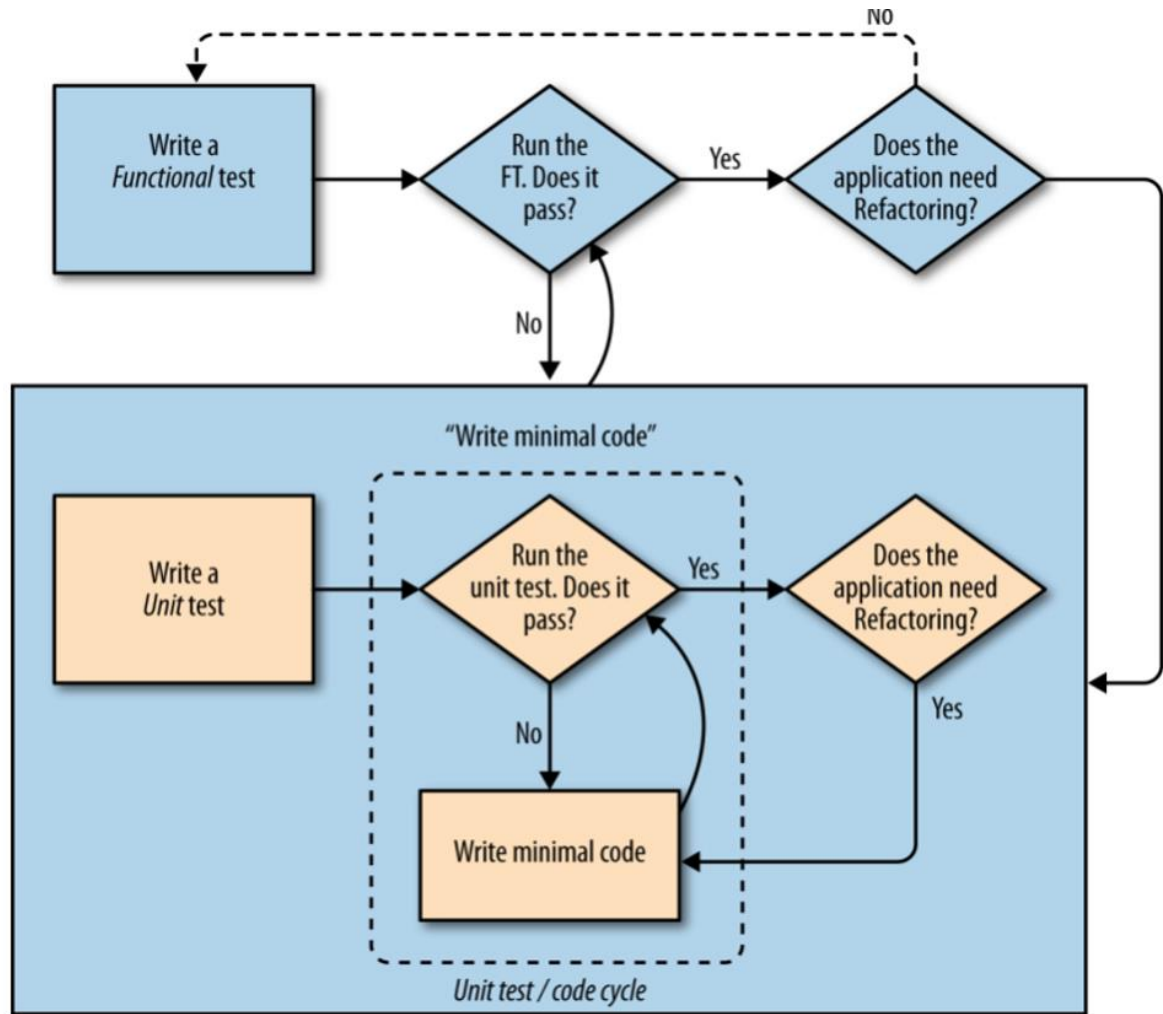
TDD Cycle

- **Red** – write a failing test
- **Green** – make the smallest change to pass the test
- **Refactor** – refactor to improve the code, e.g. remove duplicate code

TDD Cycle



Double-loop TDD (The TDD process with functional and unit tests)



Unit Test vs Functional Test

Unit Test

- Test the smallest unit (function, class, model, etc.)
- Black box – test from the outside
- White box – test from the inside
- Should have a human-readable input-output comparison
- Make it explicit using comments that accompany the implementation code. (Code documentation)

Functional Test

- User Acceptance tests, or end-to-end tests
- Look at how the whole application functions, from the outside
- Black box test, because the test doesn't know anything about the internals of the system under test
- Should have a human-readable story that we can follow
- Make it explicit using comments that accompany the test code

Unit Test

- Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually scrutinized for proper operation.
- The main objective of unit testing is to isolate written code to test and determine if it works as intended.
- Developers can perform unit tests manually or automatically.

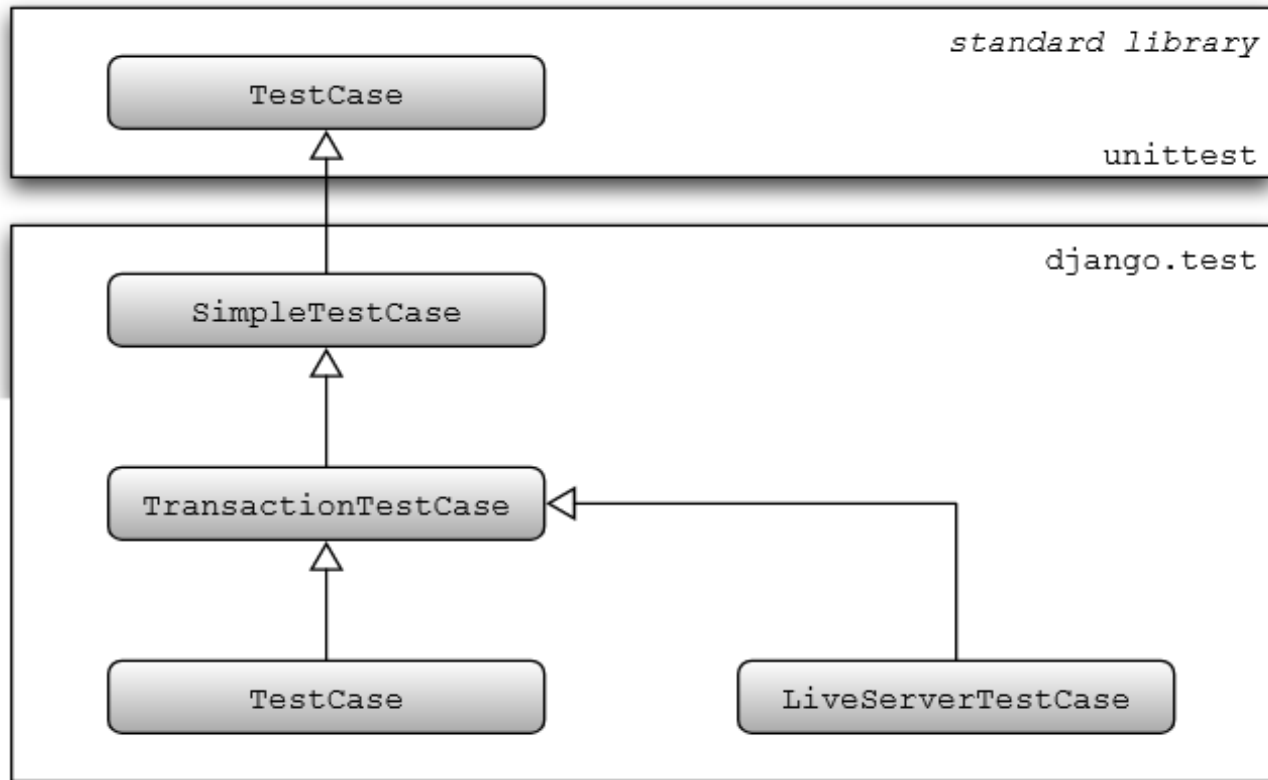
Unit Testing in Django

- Django provides a mechanism to create *unit test* – augmented version from standard *unit test* in python
- The default `startapp` template creates a `tests.py` file in the new application.

```
1 from django.test import TestCase
2
3 # Create your tests here.
4
```

Hierarchy of Django Unit Testing Class

- Normal Python unit test classes extend a base class of `unittest.TestCase`
- Django provides a few extensions of this base class
- All of the standard Python unit test functionality will be available and it will be augmented with some useful additions



Assert Methods in Python

Method	Checks that	New in
<code>assertEqual(a, b)</code>	<code>a == b</code>	
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	
<code>assertTrue(x)</code>	<code>bool(x) is True</code>	
<code>assertFalse(x)</code>	<code>bool(x) is False</code>	
<code>assertIs(a, b)</code>	<code>a is b</code>	3.1
<code>assertIsNot(a, b)</code>	<code>a is not b</code>	3.1
<code>assertIsNone(x)</code>	<code>x is None</code>	3.1
<code>assertIsNotNone(x)</code>	<code>x is not None</code>	3.1
<code>assertIn(a, b)</code>	<code>a in b</code>	3.1
<code>assertNotIn(a, b)</code>	<code>a not in b</code>	3.1
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>	3.2
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>	3.2

Assert Methods in Django

- As Python's normal `unittest.TestCase` class implements assertion methods such as `assertTrue()` and `assertEqual()`, Django's custom `TestCase` class provides a number of custom assertion methods that are useful for testing web applications, e.g `assertContains`, `assertNotContains`, and `assertTemplateUsed`.
- For other Django assert methods, see Django documentation.

Apa yang perlu dites di Django?

1. URL

- Bayangkan URL seperti apa yang diharapkan.
- Rancang juga FUNGSI apa yang akan dipanggil oleh URL tersebut.
- Buat *test*-nya terlebih dahulu.

Apa yang perlu dites di Django?

2. MODELS

- Bayangkan nama *models* yang akan dibuat, misalnya `Person` (atribut: nama dan alamat)
- Buat satu *object* untuk *model* tersebut di `tests.py`

```
Person.objects.create(nama='Bepe', alamat='JKT')
```
- Cek apakah data berhasil dimasukkan ke *database*
- Hitung jumlah *object*

```
hitungjumlah = Person.objects.all().count()  
self.assertEqual(hitungjumlah, 1)
```

Apa yang perlu dites di Django?

3. FUNGSI - VIEWS.py

- Setiap baris fungsi pada `views.py` harus dites
- Periksa setiap kemungkinan, buat *test* untuk setiap *case* yang ada, misalnya jika ada `if else`, buatlah *test* untuk `if` dan buat juga *test* untuk `else`
- Baca di dokumentasi Django, *method* Assert apa yang cocok untuk fungsi yang dibuat

Apa yang perlu dites di Django?

4. HTML terkait

- Cek apakah file HTML digunakan

```
self.assertTemplateUsed
```

- Cek apakah ada konten tertentu di HTML

```
self.assertIn
```

Functional Testing (FT) in Django

- Django also provides special support for in-browser frameworks like Selenium. For more details, see Django documentation
 - See: [LiveServerTestCase](#)
 - Basic example for tutorial app: <https://gitlab.cs.ui.ac.id/-/snippets/131>
- To install: run `pip install selenium` or add selenium to your `requirements.txt`

Example: Basic FT to Verify a Page

```
class FootballNewsFrontPageTest(LiveServerTestCase):
    @classmethod
    def setUpClass(cls):
        super().setUpClass()
        options = Options()
        cls.user_data_dir = tempfile.mkdtemp()

        if os.getenv("CI", "false") == "true":
            options.add_argument("--headless")
            options.add_argument("--disable-gpu")

        options.add_argument(f"--user-data-dir={cls.user_data_dir}")
        options.add_argument("--no-sandbox")
        options.browser_version = "139"

        cls.selenium: WebDriver = webdriver.Chrome(
            options=options, service=ChromeService(ChromeDriverManager().install())
        )

        cls.selenium.implicitly_wait(5)

    @classmethod
    def tearDownClass(cls):
        cls.selenium.quit()

        if os.path.exists(cls.user_data_dir):
            shutil.rmtree(cls.user_data_dir, ignore_errors=True)

        super().tearDownClass()
```

See complete source at
<https://gitlab.cs.ui.ac.id/-/snippets/131>

1. Initialise the test case by configuring Selenium and Django's LiveServerTestCase
 - Selenium == the 'glue' that bridges web browser with Python code
 - LiveServerTestCase == programmatically runs `manage.py runserver` in the background when executing a test case

Example: Basic FT to Verify a Page

```
def setUp(self):
    self.base_url = os.getenv("DJANGO_BASE_URL", self.live_server_url)
    print(f"Base URL for tests: {self.base_url}")

def tearDown(self):
    return super().tearDown()

def test_h1_should_contain_football_news(self) -> None:
    self.selenium.get(self.base_url)

    h1 = self.selenium.find_element(By.TAG_NAME, "h1")

    self.assertIn("Football News", h1.text)
```

See complete source at <https://gitlab.cs.ui.ac.id/-/snippets/131>

1. Initialise the test case by configuring Selenium and Django's LiveServerTestCase
 1. Selenium == the 'glue' that bridges web browser with Python code
 2. LiveServerTestCase == programmatically runs `manage.py runserver` in the background when executing a test case
2. Then, specify the base URL to be visited by the test
 1. By default, it points to <http://localhost:8000>
 2. We can configure it to point elsewhere, e.g., your app on PWS
3. Implement the test
 1. Visit the specified base URL
 2. Get the reference to the h1 element
 3. Check whether the text is correct

References

- Django documentation. Accessed: July 14, 2024. [Online]. Available: <https://docs.djangoproject.com/en/5.0/>
- R. Hamedy. "10 Reasons Why Code Coverage Matters". codeburst.io. Accessed: Jul. 14, 2024. [Online]. Available: <https://codeburst.io/10-reasons-why-code-coverage-matters-9a6272f224ae>
- TechTarget Contributor. "unit testing". TechTarget. Accessed: July 31, 2024. [Online]. Available: <https://www.techtarget.com/searchsoftwarequality/definition/unit-testing>